MATLAB 완벽가이드



저자 : 임종수

이 문서는 한글97로 제작되었습니다. 하지만 도스용 한글3.0부터 윈도우용 한글97까지 완벽히 지원됩니다. 만약 그림부분이 나타나지 않는다면 RAM이 얼만큼인지 보시기 바랍니다. RAM용량이 적으면 나타나지 않을 수 있습니다. 또한 이 글을 보시게 될 분의 글꼴이 어떤지 모르기에 가장 기본이 되는 글꼴만 이용했음을 미리 알려드립니다.
기타 문의 사항이나 더욱더 알고 싶으신분은 다음의 연락처 혹은 홈페이지에 방문하시기 바랍니다.



하이텔 : ssljk07
E-mail : matlab@loveangel.net

제작·편집 : 임종관 「오정, 한글서당, 이야기세상, MATLAB 전문학교 운영자」
http://www.matlabschool.com [MATLAB 전문학교]

10월 6, 7일 이틀에 걸쳐...
임종수씨가 진행하는 세미나 개최!!

▶ 날짜와 안내

수강 인원 : 100명
세미나 날짜 : 2001년 10월 6일 (토) ~ 10월 7일 (일)
세미나 장소 : 서울 대학로 정보세미나홀 (자세한 안내도는 홈페이지에 기재)
세미나 시간 : 오전 10시 ~ 오후 6시 (12시 ~ 1시는 점심시간, 음료수 제공)
세미나 교재 : 임종수의 MATLAB 완벽가이드
(당일 시중가의 80%에 판매 - 시중가 18,000원 → 판매가 14,400원)
세미나 방식 : 5시간 강의후 2시간 질문/답변 시간

세미나 수강료 : 학생(하루 4만원, 이틀 7만원), 일반(하루 5만원, 이틀 9만원)
수강료 납부방식 : 선불이며, 무통장입금과 신용카드결제 두가지 방식으로 진행합니다.
수강신청 : http://matlabschool.com/seminar.html

▶ 강의 내용

첫째날(10월 6일 토요일)

1. matlab overview
2. matlab의 행렬 연산.
3. M file 작성 방법.
4. Matlab 그래픽스.
5. 파일입출력과 GUI programming 기본

둘째날(10월 7일 일요일)

1. Handle Graphics system.
2. Cell and struct data type.
3. 다양한 GUI Programming 방법.

※ 이번 강의는 MATLAB 5.x에 대한 것이며, 다음에는 6.x에 대한 강의를 진행할 예정입니다.
※ 이번 강의를 수강한 모든 분들께는 다음 강의를 수강료를 할인해 드립니다.



머리말

이제부터 Matlab에 대해 강의를 할 임종수라고 합니다.

저는 현재 건국대학교 전자공학과 대학원생입니다. 여러 해 동안 Matlab을 공부해 왔으며, 헤아릴 수 없는 관련 프로그램을 작성해 본 경험과 지식을 토대로 여러분에게 조금이나마 도움이 되고자 하여 이렇게 붓을 들게 되었습니다.

앞으로, 전개해 나갈 내용은 Matlab4.0 5.1까지에 대한 내용인데, 주로 Matlab5.1에 기준을 두고 강의를 하려고 합니다. 그러나, 대부분의 학생들이 Matlab4.2c이하 버전을 가지고 있는 경우를 고려하여, 내용 전개 중에 버전사이의 차이가 있는 경우 이점을 밝힐 것입니다. 이 내용의 대상은 대학생은 물론, 고등학생도 쉽게 배울 수 있고, 이해할 수 있는 내용입니다.

앞으로, 매일 A4용지 5장 정도로 해서 Matlab의 기본을 습득하기에 충분한 내용을 전개해 나갈 것입니다.

기호 표시와 뜻			
	중요사항		메모
	주의사항		새로운 기능
	알리는 글		

I. 왜 Matlab을 배워야 하는가?

만일, 여러분에게 고등학교 수학 책에 나오는 정적분 문제, 즉,

$$y = \int_0^{\pi} \sin(x)\,dx \qquad \text{.....(1.1식)}$$

을 계산하는 프로그램을 작성하라는 과제가 주어 졌다면, 물론, 대부분의 학생은 현재 나와 있는 여러 수치 해석 책을 보고 쉽게 작성 할 수 있을 것입니다.
그러나, 만일,

$$y = \int \sin(x)\,dx \qquad \text{.....(1.2식)}$$

(1.2식)과 같은 부정 적분을 계산하여 y=-cos(x)와 같은 결과가 나오고, 그 결과를 그림으로 나타내보라고 하면, 문제는 달라 질 것입니다. 설령, 할 수 있다고 하더라도 상당한 시간을 요할 것입니다. 그러나, Matlab에서는 (1.2식)과 같은 문제는 간단히 해결됩니다. 즉,

```
□y=int('sin(x)');
□ezplot(y);
```

라고 하면 되기 때문입니다. 이처럼 Matlab은 강력한 수학적인 능력을 갖고 있고, 또한, 쉽게 주어진 상황을 코딩화 할 수 있습니다.
그 동안, 여러분은 mathematica 또는 mathcad와 같은 여러 수학관련 도구(tool)를 들어 보았을 것입니다. 필자도, 처음에는 mathcad, mathematica를 다루어 보았지만 Matlab만큼 강력하지는 않았습니다. 지금 현재 서방 여러 나라에서도 모든 코딩이 거의 다 Matlab으로 방향을 전환하려는 추세인 것만 봐도 그 강력한 능력을 알 수 있을 것입니다.

II. Matlab은 무엇인가?

원래 Matlab은 Cleve Moler에 의해 Fortran으로 작성되었으나, 현재는 미국의 MathWorks 사에 의해 C++로 작성되었습니다.

Matlab 코딩 체계는 우리에게 친숙한 수학적인 기호와 간단한 C문법으로 행해지는데, 전형적인 이용 범위는 다음과 같습니다.

① 수학과 관련된 계산
② 알고리즘 개발
③ 상황 모델링과 data분석
④ 여러 가지 과학과 공학적인 그래픽적 표현
⑤ GUI(Graphical User Interface)에 의한 에플리케이션 개발

Matlab의 기본 데이터 요소는 차원의 제한이 없는 array(배열)입니다. 즉, 기존의 High-level 언어에서는 행렬 연산의 경우, 연산에 대한 해당 함수를 만들어 주어야 했지만 Matlab에서는 기본 피 연산자가 array이므로 한번에 일괄적으로 계산할 수 있습니다.


주의

Matlab 4.2c까지는 array가 2차원적인 경우만 되었다.
그러나, Matlab5.0부터는 다차원 배열이 가능하다.

Matlab은 이용하고자 하는 분들의 전공에 도움을 주고자 toolbox를 가지고 있습니다. toolbox란 신호 처리, 통계학, 영상 처리, 제어, fuzzy logic, 재정, 화학 공정, 등등, 즉, toolbox란, 해당 전공 부분의 내용을 심도 있게 지원하는 함수들의 도서관과 같은 것입니다. 특별히, simulink라는 것이 있는데 이 디렉토리의 내용은 주로 동적 시스템의 simulation에 이용되는 부분입니다.

Matlab은 외부 프로그램, 즉, C, Fortran과 link해서 이용할 수 있는 기능도 제공하고 있습니다.

III. installation

Matlab은 PC뿐만 아니라, UNIX, MAC.에서도 사용 할 수 있으며 이용 방법은 거의 같다고 보아도 됩니다. 필자도 PC, UNIX를 번갈아 가며 Matlab을 이용하여 보았지만 거의 용법의 차이를 보지 못했습니다.

여기서는 window95에 기반을 둔 PC의 경우만 예를 들겠습니다.

가. 기본 사양:

1. Intel 486 프로세서이상으로 math coprocessor를 포함한 것 이상 일 것.
   → 486DX이상은 math coprocessor가 포함되어 있으므로 상관없지만, 만일, 독자가 GUI를 이용한 응용 프로그램을 작성하고자 원한다면 필자의 경험상 실행 속도 때문에 Pentium133이상은 되어야 한다고 봅니다.
2. RAM은 16MB이상은 되야 정상적으로 영상 처리등을 이용할 수 있습니다.
3. Help file에는 HTML문서와 .PDF문서가 있는데, .PDF 문서는 CD에 포함되어 있는 Adobe Acrobat Reader를 이용하거나, www.adobe.com에 접속하여 관련 정보를 얻을 수 있습니다.
   → .PDF문서에는 Matlab에 관한 reference가 잘 정리되어 있으므로 꼭 한번 살펴보길 바랍니다.

나. installation :

1. Matlab CD는 CD-ROM에 넣으면 자동 실행되지만, 안 되는 경우, setup.exe icon을 double-click해도 됩니다.
   installation이 실행되면 초기 화면이 다음과 같이 나올 것이다.

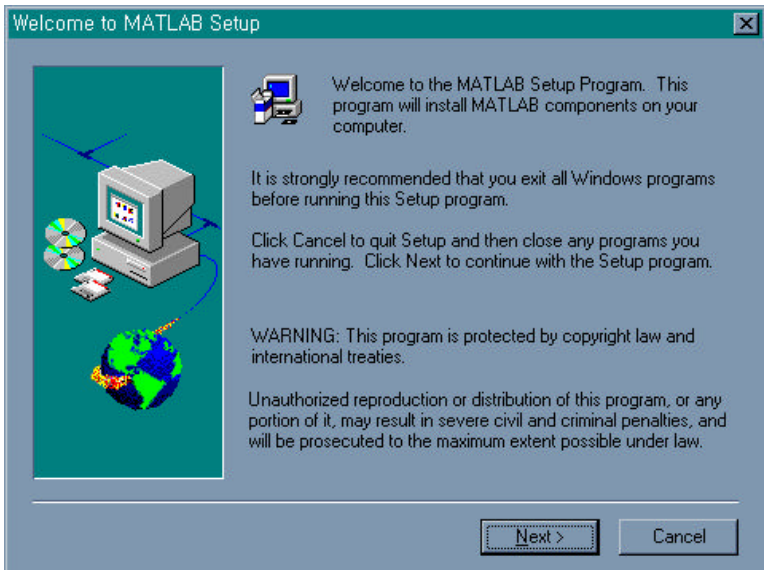


그림 1.1 matlab installation

2. software license에 동의한다고 하면 다음과 같은 [그림 1.2]가 나올 것이다.
여기서, install하고자 하는 버전을 선택하고, network에 연결해서 사용할 것인지를 결정한다. license번호를 정확히 기입한 후 next 버튼을 누르면 다음의 화면으로 들어간다.

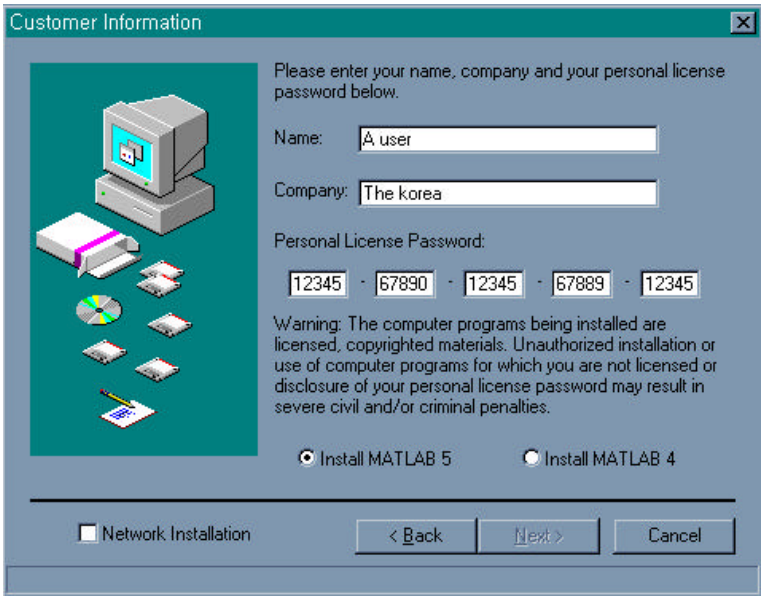


그림 1-2. matlab installation

3. [그림 1.3]에서는 여러분이 install하고자 하는 Matlab성분을 선택해야 하는데, 필자가 추천 해주고 싶은 공통 부분은 Matlab, Matlab tour, simulink, symbolic math, statistical, signal processing, image processing, control이다. 이중에서 대부분의 학생들이 symbolic math를 안 설치하는데 그것은 상당히 잘못된 생각이다.

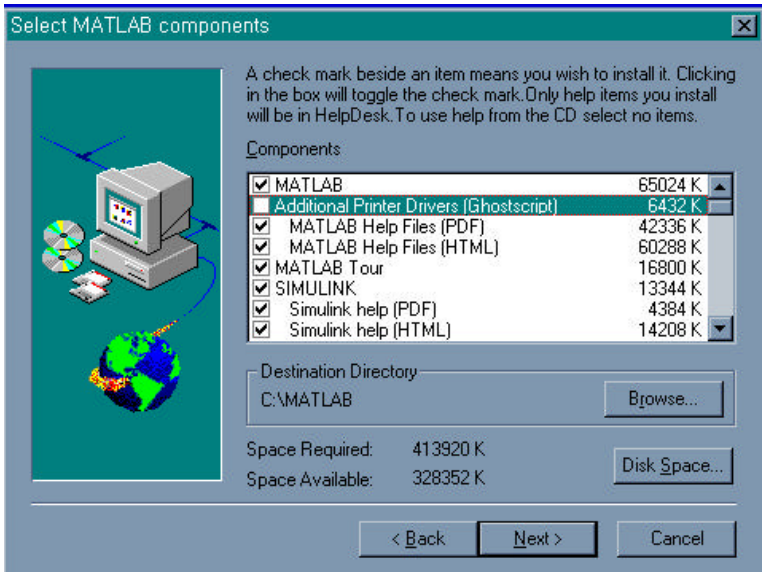


그림 1.3 Matlab installation

symbolic math란 일종의 변수 자체를 계산하는 것으로, 일종의 부정 적분과 같이 답이 변수로 나오게 하는 것이다. MathWorks사가 Maple사에 license를 주고 사온 maple 프로그램이 이곳에 설치되어 있으므로 만일, 독자중에서 Maple을 다루어 본 적이 있는 분에게는 상당히 유용한 toolbox가 될 것이다. 또한, statistical toolbox는 비록 통계학 전용 tool인 SAS에 비해서는 다소 기능이 떨어지지만, 본인도 수리 통계학을 공부 해본 결과, 아무런 불편 없이 자료를 분석할 수 있었습니다.

optimization toolbox는 비 선형 정합 문제를 다루는 것으로 toolbox중 ident. toolbox 다음으로 수학적인 지식을 많이 필요로 하는 toolbox이다.

여기서, Matlab toolbox는 따로 사는 것이라는데 주의하자!

Matlab toolbox는 해당 분야에 권위 있는 대학교수나 전문가를 초빙하여 알고리즘을 개발하여 만든 것으로, 각각의 해당 toolbox 매뉴얼을 보면 여러분은 전공도 배우고 시뮬레이션도 배우는 일석이조의 기쁨을 얻을 수 있다.

4. [그림 1.4]에서는 독자에게 MS word를 갖고 있는 경우, notebook을 설치할 것을 권하는 문장이 나온다.

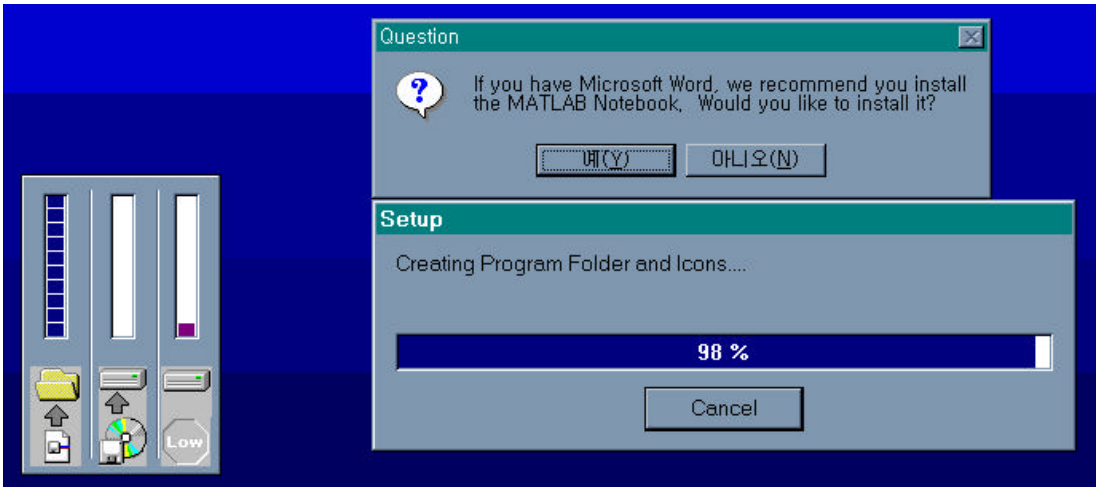


그림 1.4 Matlab installation

Matlab은 여러분이 MS word를 이용하여 문서를 작성하는 경우, 문서 내에서 작성한 수식을 Matlab이 계산하여 그 결과를 MS word에 곧 바로 나타내주는 기능을 갖고 있다. 즉, Matlab 코딩체계를 MS word에서 하면, 그 결과가 MS word에 나온다는 것이다. 만일, notebook을 설치하면 MS word에 추가된 icon을 볼 수 있을 것이며, M-file이라는 새로운 text양식이 추가 된 것을 볼 수 있다. 그러나, 필자의 생각으로는 설치하지 않는 것을 권한다. 그 이유는 MS word로 문서를 작성하고, Matlab으로 계산하려면, 결국, word와 Matlab을 동시에 실행해야 하므로 자원의 낭비가 심하며, 무엇보다, debuging과정이 쉽지 않다.

→ UNIX, Window NT, MAC.에서도 installation이 어렵지는 않을 것이다.

다. 경로 설정 :

여러분이 Matlab을 성공리에 installation을 한 뒤, Matlab을 실행시키면 다음과 같은 화면이 나올 것이다.

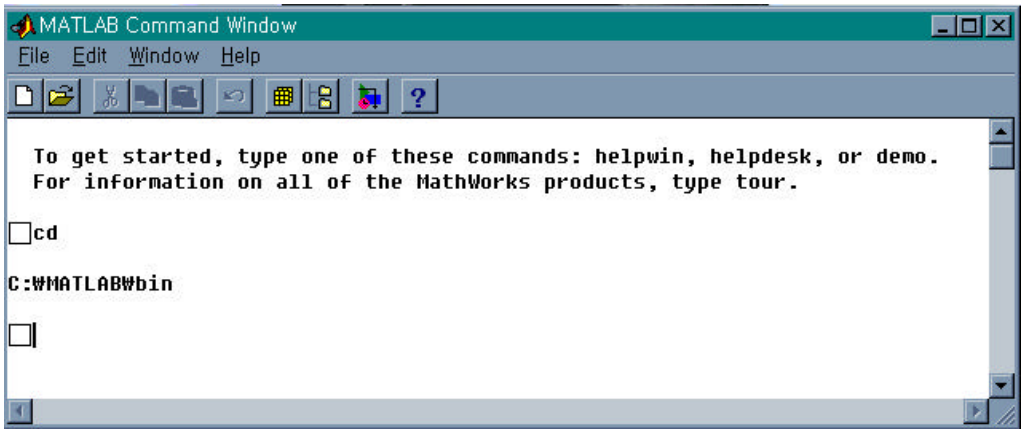


그림1.5 Matlab 실행시의 화면

[그림1.5]에서 알 수 있듯이, Matlab이 실행되면 초기 작업 디렉토리가 "c:\Matlab\bin"으로 결정된다. 결국, 여러분이 앞으로 작성할 모든 Matlab파일은 전부 "c:\Matlab\bin"에 저장되므로 원래 "c:\Matlab\bin"에 있던 파일과 혼동을 할 우려가 있다. Matlab4.2c이하 버전에서는 "Matlabrc.m"이라는 파일이 있어서 이곳에 모든 Matlab경로와 환경을 사용자가 설정할 수 있도록 해주었다. 그러나, Matlab5.0이후 버전에서는 이와 같은 경로와 환경설정을 보다 간편히 할 수 있도록 새로운 "path browser"라는 tool을 제공해준다. 우선 "c:\Matlab\toolbox\local"디렉토리에 있는 "Matlabrc.m"이라는 파일을 열어서 맨 아래 부분을 보면, 다음과 같은 내용을 볼 수 있을 것이다.

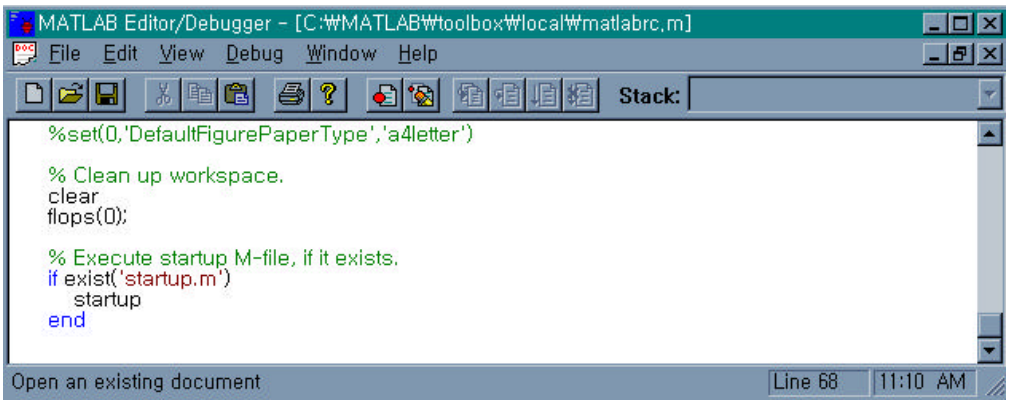


그림 1.6. Matlabrc.m파일의 맨 아래 부분의 내용

[그림 1.6]의 내용은 "만일, Matlab이 초기 실행 될 때, "startup.m"이 있으면 그 내용을 우선 실행하라는 뜻"이다. 그러므로, 여러분은 우선, "stratup.m"이라는 파일을 만들어서 그곳에

여러분이 원하는 경로나 환경을 설정 해줄 수 있다. 사실, “Matlabrc.m"내에서 환경을 설정 해 줄 수도 있지만, "Matlabrc.m"의 내용은 편집하지 않는 것이 좋다. [그림1.7]에서는 간단한 경로만 설정해주는 "startup.m"파일을 보여주고 있다.

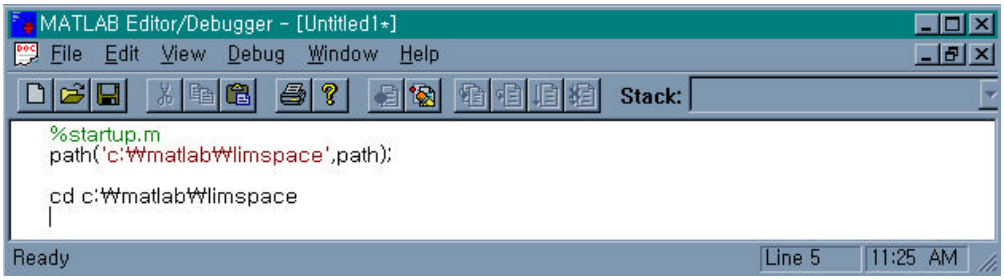


그림 1.7. startup.m파일의 내용

이제, "stratup.m"을 Matlab의 “bin"디렉토리에 저장한 후, ”c:\Matlab\limspace"디렉토리를 새로 만들고, Matlab을 다시 실행하여 [그림1.5]와 같이 명령어 ”□cd"을 행하면 □ c:\Matlab\limspace으로 작업 디렉토리가 옮겨 진 것을 알 수 있을 것이다. Matlab을 installation 한 후에 “*.m file"의 icon을 보면 아직 link될 프로그램이 설정 되어 있지 않은 것을 알 수 있다. 그러므로, file 탐색기의 옵션 메뉴를 이용하여 “*.m file"을 Matlab에 link 시켜주도록 하자. 여러 가지 link시켜주는 방법이 있겠지만 여기서는 그 중에서 하나의 방법을 보여 주겠다.

1. 임의의 “*.m file"을 mouse 왼쪽 키로 한번 클릭한다.
2. file 탐색기의 보기 메뉴에서 “옵션”을 선택한다.
3. 옵션에서 “파일 형식”을 선택한다.
4. “새 형식”을 클릭한다.
5. “파일 형식 설명”란에 “m-file"이라 쓴다.
6. “연결된 확장명”에는 “.m"이라고 쓴다.
7. “새 명령”을 누른다.
8. “명령”에는 “open with Matlab"이라고 쓴다.
9. “명령을 실행할 응용 프로그램”에는 옆에 있는 “찾아 보기”를 눌러서 Matlab.exe를 찾아 더블 클릭한다.
10. “확인”을 클릭하고, 이어서 “아이콘 바꾸기”로 Matlab icon으로 바꾸어 준다.
11. “확인”을 클릭하면, “파일 형식 정보”에 여러분이 바꾸어 준 정보가 나올 것이다. 그리고, 동시에 “*.m file"파일의 icon들이 여러분이 설정한 icon으로 모두 바뀌는 것을 알 수 있을 것이다.

이제, “*.m file"파일은 클릭한 장소에 관계없이 항상 Matlab을 실행시킬 것이며, 동시에 ”c:\Matlab\limspace"로 작업 디렉토리를 설정 해줄 것이다.

지금까지 여러분은 Matlab의 기본 installation과정과 간단한 경로 설정 방법을 배웠다. 다음 단원부터는 본격적으로 Matlab을 이용한 수치 계산과 그에 따른 그래픽 표현을 배우게 될 것이다.

1 행렬연산

I. Matlab의 행렬연산 방법.

원칙적으로 행렬(matrix)과 배열(array)은 의미상 차이가 있지만, 편의상, 앞으로 이들 행렬과 배열은 같은 의미로 생각하겠다.
기본적으로, "= (즉, assignment operator)"왼쪽에는 저장할 변수(variable), 오른쪽에는 저장할 행렬수식으로 이루어진다. 즉,

□변수명(variable) = 수식(expression)

여기서, 변수란 double형, char형, cell형, struct형, sparse형, user_defined형을 말하며, 수식이란, Matlab이 해석할 수 있는 numeric and nonnumeric형의 수식을 말한다.


주의 변수의 형(type)중에서, cell형과, struct형, user_defined형은 Matlab5.0이후 버전에서만 지원된다. 또한, 다차원 배열도 Matlab5,0이후 버전에서만 지원된다.

* Matlab에서 행렬을 만드는 방법은 다음과 같다.

1. 명확한 행렬요소를 작성한다.
2. 외부 데이터 파일로부터 행렬을 load한다.
3. M-files로 행렬을 생성한다.
   → 이번 단원에서는 1.번만 알아보고, 2.,3.번 내용은 추후에 알아보도록 한다.

가. 행렬의 합과 transpose

Matlab으로 행렬을 만드는 경우, 열(column)의 원소들은 blanks또는 commas로 구분을 한다. 행(row)의 원소들은 semicolon(즉, ;)으로 구분하거나 "enter 키"를 누르면 된다. 다 작성한 행렬요소는 square brackets(즉, [ ])으로 닫아 준다. 예를 들어서, 3행3열의 다음과 같은 행렬 A를 만들어 보자.



앞으로 참고 해야할 내용은 "%"기호 다음에 작성할 것이다. Matlab은 주석을 "%"기호 다음에 쓰게 되어있다. 또한,
" ↲ "기호는 "enter 키"를 누르라는 뜻이다.

$$A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 6 \end{bmatrix} \quad \text{..... (1.1식)}$$

□A =[1 1 1;1 2 3;1 3 6] ↲
□A =

A =

1 1 1
1 2 3
1 3 6

이제, 행렬A의 합을 구해 본다. 기본적으로 Matlab의 모든 연산과 함수의 입력 data처리는 "열 벡터" 방향이다. 즉,

□sum(A) ↲ 를 하면 결과는 각각의 열을 더한 결과가 행 벡터 형태로 나온다.

ans =

3 6 10

여기서, 여러분은 재차 Matlab이 "열 벡터"방향으로 연산한다는데 주의해야 한다.

Matlab은 출력 변수를 설정하지 않은 경우 변수 "**ans**(answer의 약식)"에 값을 할당한다. 또한, " □y=sum(A); "와 같이 문장 맨 뒤에 semicolon을 쓰면 출력 결과가 표시되지 않는다. 행렬의 transpose(i×j행렬을 j×i행렬로 바꾸는 연산자)에는 두 가지가 있다.

1. 복소 벡터나 행렬 Z의 " □**Z'** "은 복소 공액(complex conjugate) transpose를 나타낸다.
2. 복소 벡터나 행렬 Z의 " □**Z.'** "은 일반적인 행렬의 transpose를 나타낸다.

→ 결국 " **.** (즉, dot)"가 있는가 없는가에 따라 뜻이 달라진다.

```
□z=[1+2i 3+4i] ↲

z =

   1.0000 + 2.0000i   3.0000 + 4.0000i

□z' ↲

ans =

   1.0000 - 2.0000i
   3.0000 - 4.0000i

□z.' ↲

ans =

   1.0000 + 2.0000i
   3.0000 + 4.0000i
```

이 된다.

이밖에 다른 여러 가지 연산, 예를 들어서 "cos 함수"에 대한 이용방법을 알려면

□help cos ↲ %즉, " help topic_name "의 양식!

위와 같이 하면 자세히 그 함수에 대한 설명이 나온다.

만일, 여러분이 " **fft** "에 관련된 여타 함수들의 정보를 알고 싶다면,
" □lookfor topic_name "이라고 하면 된다. 즉, " help "는 주어진 topic에 관한 내용만 제공해주고, " lookfor "는 Matlab이 지원하는 모든 함수에 대한 정보를 제공해 준다. (단, "path browser"에 등록된 경로내의 함수들만 해당된다. 그러나, "overloaded"된 함수나 연산자는 정보를 제공해준다.)

□lookfor fft ↲
FFT Discrete Fourier transform.
FFT2 Two-dimensional discrete Fourier Transform.
FFTN N-dimensional discrete Fourier Transform.
FFTSHIFT Shift DC component to center of spectrum.
IFFT Inverse discrete Fourier transform.
… 생략 …

여기서, "help"와 "lookfor"에서 제공되는 함수 정보는 M-file의 주석 내용들이다.
(단, "lookfor"의 경우에는 주석문의 제일 윗줄만 해당된다.)

이제, 여러분이 만든 변수와 데이터에 대한 정보를 알아보자. 현재, A라는 3×3행렬과 Z라는 복소 행렬 그리고 출력 변수를 정하지 않아서 생긴 " ans "행렬 이렇게 3가지를 만들었다.
데이터에 대한 정보를 알려면, " **whos** "라는 명령어를 사용하면 된다.

□whos ↲

Name	Size	Bytes	Class
A	3x3	72	double array
ans	3x1	24	double array
z	1x2	32	double array (complex)

Grand total is 14 elements using 128 bytes

위와 같은 데이터의 정보가 나오는데, 만일, " ans "행렬을 base workspace(Matlab의 command window에서 행한 변수를 저장하는 메모리 공간, 이밖에도, 함수 workspace, callback workpsace가 있다.)에서 지우려면,

□clear ans ↲

하면 " ans "변수가 지워진다. 또는, Matlab의 "workspace browser"를 이용한다.

[그림1.8] Matlab command window toolbar를 보여 주고 있다. 여기서, "workspace browser" icon을 click한다.



그림 1.8 command window toolbar

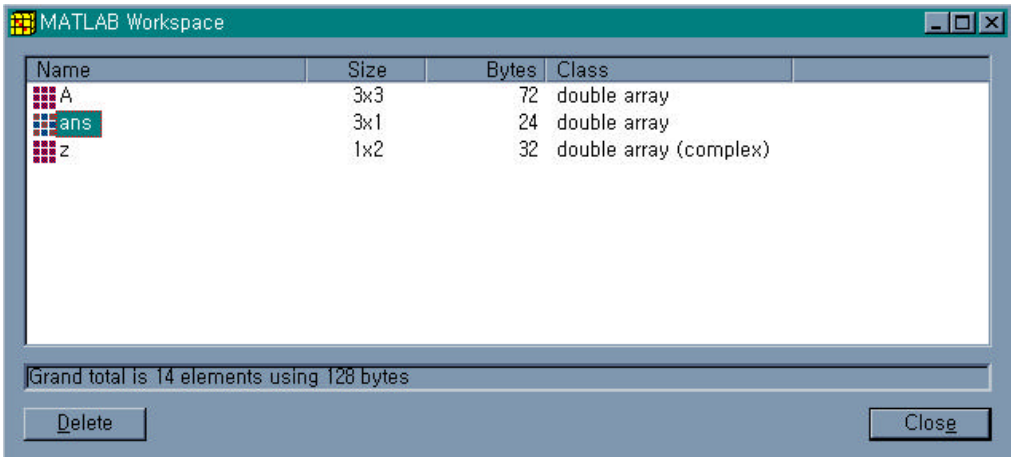


그림 1.9 Matlab workspace browser

[그림 1.9]는 “workspace browser"를 이용한 예를 보여 준다.

나. 행렬을 구성하는 원소들의 참조 방법.(첨자:subscript)

행렬의 원소들은 임의의 Matlab식이 될 수 있다. 여기서, 말하는 식이란 Matlab함수들도 포함된다.

```
□x=[-1.3 sqrt(3) (1+2+3)*4/5] ↲
x =

   -1.3000    1.7321    4.8000
```

“ x "개개의 성분을 얻으려면 ” ( ) “안에 해당 원소의 위치(subscript)를 넣어 주면 된다.

```
□x(3) ↲
ans =

    4.8000
```

그런데, 만일, 다음과 같은 명령을 주면 어떻게 될까?

```
□x(5)=abs(x(1)) ↲
 x =
   -1.3000    1.7321    4.8000         0    1.3000
```

결국, " X "의 크기는 새로운 5번째 원소를 수용하기 위해서 증가해버린다. 그리고, 4번째 원소는 " 0 "으로 채워진다. 앞으로 배울 다차원 행렬뿐만 아니라, cell형이나 struct형들도 새로운 데이터를 수용하도록 자동으로 증가하고, 빈 부분에는 " 0 "이나 빈 행렬(empty matrix)로 채워준다는 사실을 알게 될 것이다.

작은 행렬로 큰 행렬을 만드는 경우도 있다.

```
□x=[-1.3 sqrt(3) (1+2+3)*4/5]; ↲
□y=pascal(3); ↲   % "pascal(3)"는 고3수학 책에 나오는 pascal의 삼각형을
                  % 구해주는 함수이다.
□xy=[y ; x]; ↲
```

여기서, " xy "는 어떻게 출력될까? 잠시 생각해 보기 바란다.

semicolon은 새로운 행(row)를 만들어주는 연산자라고 하였다. 그렇다면, " y "가 3×3행렬이므로 4×3행렬이 될 것이다. 원소간의 배치는 " y "행렬 다음에 " x " 벡터가 나오므로 " xy "행렬은 다음과 같이 된다.

```
□xy ↲
xy =

    1.0000    1.0000    1.0000
    1.0000    2.0000    3.0000
    1.0000    3.0000    6.0000
   -1.3000    1.7321    4.8000
```

그러나, " □xy=[y ; x'] ↲ "를 하면은 어떻게 될까?
" y "는 3×3행렬인데, 새로운 x' 3×1 열 벡터가 " y "행렬 밑에 놓이는 모양이 되므로 행렬의 모양을 이루지 못하여 " error "가 발생한다.

```
□xy=[y ; x'] ↲
??? All rows in the bracketed expression must have the same
number of columns.
```

다. The colon operator.

Colon(즉, " : ")은 Matlab연산자중에서 가장 중요한 연산자중에 하나이다. 사실, Matlab coding을 얼마다 효율적으로 했는가는 이 " colon "의 사용 방법에 달려 있다고 보아도 과언이 아니다.

```
□x=1:10 ↲
x =

     1     2     3     4     5     6     7     8     9    10
```

" x "가 1부터 하나씩 증가하여 10까지 증가한다. 만일, 최소값을 "min_val", 최대값을 "max_val"이라고 하고, 증분을 "delta"라고 하면 다음과 같은 형태를 취한다.

```
□x=min_val : delta : max_val ↲
```

Colon 연산자는 행렬의 subscript로도 사용할 수 있다.

```
□x=-3:-1; ↲
□y=pascal(3); ↲
□y(1,1:3)=x ↲    % " □y(1:3,1)=x "라고 하면 "error"가 발생한다. 그 이유는 ?
y =

    -3    -2    -1
     1     2     3
     1     3     6
```

" y "의 1행 원소가 모두 바뀌었다.



처음 Matlab을 배우려는 대다수의 학생들은 행과 열의 조작에 무척 당황해 하고 지겨워한다. 그것은 필자도 마찬가지였다. 그러나, 행과 열의 자연스러운 조작 기법을 배우는 것이, 비단, Matlab만을 위한 것보다는 행렬계산의 능률면에서 중요한 도구가 된다.

새로 만들어진 " y "행렬 모든 원소의 summation(합)을 구해 보자!

```
□sum_y=sum(sum(y)) ↲
 sum_y =

     10
```

Matlab에는 " **end** "라는 연산자가 있다. 이 연산자는 " for, while, switch and if "와 연결되어 C언어에서와 같이 제어의 흐름을 조종하기도 하고, 행이나 열 벡터의 마지막 index를 나타내주기도 한다.

```
□sum_r=sum(y(end,:)) ↲
sum_r =

    10
```

여기서, " 10 "은 " **y** "의 마지막 행(1, 3, 6)의 합이다.

```
□sum_c=sum(y(:,end)) ↲
sum_c =

     8
```

여기서, " 8 "은 " **y** "의 마지막 열(-1, 3, 6)의 합이다.

이제, " **y** "의 평균을 구해 보자!

```
□mean_y=sum(sum(y))/prod(size(y)) ↲  % " prod "함수는 원소들간의 곱을 반환,
mean_y =                              % " size "함수는 행렬의 크기를
                                      % 행 벡터의 모양으로 반환한다.
    1.1111

□[r,c]=size(y) ↲
r =

     3

c =

     3
```

다음과 같은 행렬의 조작법도 있으니 잘 숙고 해 보자!

```
□B=magic(4) ↲
B =
    16     2     3    13
     5    11    10     8
     9     7     6    12
     4    14    15     1
□A=B(:,[1 3 2 4]); ↲
```

과연, 이 명령이 무엇을 뜻할까?

해석 해보면, " 행렬 B의 각각의 행들에 대한 열들을 1, 3, 2, 4로 재배치하라는 뜻이다."

```
□A ↲
A =

    16     3     2    13
     5    10    11     8
     9     6     7    12
     4    15    14     1
```


주의 변수의 이름을 만일 " cos "이라고 하면 앞으로 " cos "함수는 사용할 수 없다는데 주의하라! 다시 말해서, 함수의 이름을 변수 이름으로 쓰면 그 함수는 더 이상 재 기능을 못하므로 변수의 이름을 정할 때나 M-files의 이름을 정할 때는 각별히 주의해서 사용 해주기 바란다.

지금까지 Matlab에서 행렬을 조작하는 간단한 방법을 배웠다. 주의 할 것은 [그림 1.8]과같은 "command window toolbar"는 Matlab5.0이후 버전에만 있다는 것을 명심하자! 다음 단원「라」에서는 보다 구체적인 행렬 연산에 대해 학습한다.

라. Matlab에 의한 행렬 연산.

1. 행렬의 덧셈과 뺄셈 :

Matlab은 행렬을 하나의 기본 피 연산자로 간주한다. 그러므로, 행렬의 덧셈과 뺄셈도 마치 스칼라(scalar)간의 연산과 같이 간단히 표현된다.

```
□A =pascal(3); ↲
□B =magic(3); ↲  % pascal 과 magic등은 Matlab이 지원하는 일종의 데모 행렬.
□Y =A +B ↲       % Matlab은 변수의 이름에 대한 대·소문자를 구분한다.

Y =
     9     2     7
     4     7    10
     5    12     8
```

뺄셈도 마찬가지 방법으로 행한다. 그러나, 두 개의 피 연산자는 같은 크기의 차원을 가지거나, 또는 , 스칼라(scalar)이어야 한다.



만일, 여러분이 help topic_name**을 행한 경우, 간혹, 어떤 함수는 입력매개변수로 "**DIM"**이라고 하여 차원을 입력하게 하는 것이 있다. 이때, "**DIM=1"**인 경우 는 열 벡터를 의미하고, "**DIM=2"**인 경우 행 벡터를 의미한다. 이것은 Matlab**이 열 벡터 위주의 계산을 행하기 때문이다. 여러분이 신호 처리** toolbox**를 이용할 때 이 문제는 중요한 관건이 된다.**

```
□C=ones(2,3); ↲     % "ones"함수는 모든 원소가 "1"인 입력 매개변수 크기의 행렬
                    % 을 만든다.
```

```
□X=A+C ↲
??? Error using ==> +
Matrix dimensions must agree.
```

두 개의 피 연산자의 크기가 행렬의 덧셈 상 맞지 않다. 앞으로, 여러분이 M-files**를 만드는 경우 위와 같은** error**는 빈번히 발생 할 것이다.**

```
□X=A+2 ↲

X =

    11     4     9
     6     9    12
     7    14    10
```

행렬에 scalar**를 더하거나 빼는 경우, 이것은 반대편 행렬의 모든 원소에 그** scalar**를 더하거나 뺀 효과가 된다.**

2. 행렬의 곱셈과 나눗셈 :

```
□A =pascal(3); ↲
□B =ones(3,2); ↲
□C =1:3; ↲
□D =2:4; ↲
□X =A *B ↲

X =

     3     3
     6     6
    10    10
```

위와 같이 " X "는 행렬 " A "와 " B "의 크기가 곱셈 상 맞으면, 곱셈이 행해 진다.

```
□X =C*D ↲

??? Error using ==> *
Inner matrix dimensions must agree.
```

그러나, 위와 같이 곱셈 상 크기가 맞지 않으면, error**가 발생한다. 그런데, 곱셈 기호 " * " 대신에 " .* "를 이용하면 다음과 같은 결과가 생긴다.**

```
□X =C.*D ↲

X =

     2     6    12
```

결과를 잘 살펴보면, C**는** [1 2 3]**이고,** D**는** [2 3 4]**일 때, 연산 결과** X**는** [2 6 12]**이므로 " .* "은 원소 대 원소(**element-by-element**)곱셈인 것을 알 수 있다. 그러므로 연산자 " .* "은 두 개의 피 연산자인 행렬들의 크기가 같기만 하면 원소 대 원소의 곱셈을 행한다. 같은 방식으로 나눗셈을 행할 수 있다.**

```
□X =C/D ↲ %여기서 대부분의 학생은 벡터 대 벡터의 나눗셈이 가능한가 의문을
          %가질 것이다.

X =

    0.6897
```

그러나, 결과는 계산이 가능하며 그 값은 0.6897이라고 나왔다. 사실, 행렬간의 나눗셈은, 이 경우 벡터" D "가 역 행렬을 가져야 된다. 그러나, " D "는 벡터로 역 행렬을 갖질 수 없다. 그런데도 계산 결과가 나왔다. 그 이유는 무엇일까?

필자의 생각으로는 독자가 만일, 공학 석사 수준이라면 왜 계산 결과가 나왔는지 유추 할 수 있다고 본다. Matlab은 " D "의 역 행렬을 구한 것이 아니라 의사 역 행렬(pseudoinverse)구하여 계산한 것이다. 의사 역 행렬을 간단히 설명하면, 미지수에 비해 방정식이 많을 때, 우리는 least squares soultion을 구한다. 이때, 마치, 역 행렬과 같은 기능을 하는 행렬이 만들어지는데 이 행렬을 의미한다. 사실, 이 행렬은 overdetermined system의 해를 구하는 데 필요한 행렬이다. 만일, 시간이 허락 한다면 시스템 론과 간단한 선형 대수학을 논할 것이다.


주의

"JOHN WILEY & SONS, INC."에서 1996년 출간한 "statistical digital signal processing and modeling"의 page33을 보면 pseudoinverse문제를 잘 설명하고 있다. 또한, "Prentice-Hall International, Inc."에서 1995년 출간한 "Lessons in estimation theory for signal processing, communication, and control"의 단원 4를 보면 더욱 일반적인 내용까지 잘 설명이 되어 있다.

Matlab에서는 "pinv"라는 의사 역 행렬을 구해주는 함수가 있다.

```
□inv_D=pinv(D); ↲
□new_X=C * inv_D ↲

new_X =

    0.6897
```

위에서 얻은 " X "와 같은 결과를 얻는다. " / "은 원소 대 원소의 나눗셈을 행한다.

```
□X=C ./ D ↲

X =

  0.5000    0.6667    0.7500
```

3. 행렬의 거듭 제곱 :

곱셈과 나눗셈처럼 거듭제곱에도 두 가지 연산자가 제공된다. 즉, " ^ "과 " .^ "

```
□A =pascal(3); ↲
□Y =A^2 ↲

Y =

     3     6    10
     6    14    25
    10    25    46
```

위의 결과는 □Y =A *A와 같다.

```
□X =A .^2 ↲

X =

     1     1     1
     1     4     9
     1     9    36
```

위의 결과는 행렬 A의 각 원소를 거듭 제곱한 것과 같다. 이밖에도 "sqrtm"과 "expm"과 같은 함수가 있다.

```
□X =sqrtm (A ) ↲

X =

    0.8775    0.4387    0.1937
    0.4387    1.0099    0.8874
    0.1937    0.8874    2.2749
```

위의 결과는 행렬의 고유치(eigenvalues)의 분포에 좌우된다.

□Y=sqrt(A) ↲

X =

1.0000	1.0000	1.0000
1.0000	1.4142	1.7321
1.0000	1.7321	2.4495

위의 결과는 행렬A의 각 원소에 대한 square roots를 구한 것이다. 결국, "sqrtm(A)"함수는 "A^(1/2)"와 같은 의미이고, "sqrt(A)"함수는 "A.^(1/2)"와 같은 의미이다.

□C=1:3; ↲
□D=2:4; ↲
□Z=2.^[C D] ↲

위의 결과는 어떻게 나올지 생각해 보기 바란다.

Z =

2	4	8	4	8	16

독자가 생각한데로 잘 나왔는가? 그렇다면 다음 단원으로 넘어간다.

4. 여러 가지 특수 행렬 함수들:

가) 단위행렬(또는, 항등 행렬, identity matrix) :

단위 행렬을 만들어 주는 함수는 " **eye** "이다.

□iden=eye(3) ↲

iden =

1	0	0
0	1	0
0	0	1

나) 빈 행렬(empty matrix) :

만일, 3×3행렬에서 2행의 모든 원소를 없애 버리려면 어떻게 해야 할까?

□A=pascal(3) ↲

```
A =

     1     1     1
     1     2     3
     1     3     6
```

□A(2,:)=[] ↲

```
A =

     1     1     1
     1     3     6
```

위와 같이 제거하려는 행이나, 열에 "[ ]"을 대입하면 된다.
matlab에서는 0×0행렬 즉, 원소가 없는 행렬만 empty matrix라고 하지는 않는다.
m×0×n행렬처럼 어느 하나의 벡터가 없는 것도 모두 empty matrix라고 부른다.

다) 행렬의 판별식(determinant)의 값을 구하는 함수:

행렬의 판별식의 값은 "det"라는 함수로 구할 수 있다.

□A=pascal(3); ↲
□det_A=det(A) ↲

```
det_A =

     1
```

그밖에도, 다음과 같은 행렬이 있다.

inv(A)는 행렬의 역 행렬을 구한다.
eig(A)는 행렬의 고유치와 고유 vector를 구한다.
jordan(A)는 행렬의 jordan형을 구한다.
svd(A)는 행렬의 singular value decomposition을 행한다.

5. matlab이 표현하는 데이터의 format

"format"명령어는 matlab에 의해 표현되는 수치의 형태를 제어한다.



"format"명령어는 단지 screen에 표현되는 데이터의 표현 양식만 바꾸어 주는 것이지 결코, 계산상의 정밀도를 바꾸어 주는 것은 아니다. matlab의 모든 계산상의 정밀도는 " double 형 (8바이트, 정확히 15자리)"에 기준을 두고 있다.

이제부터, 2개의 원소를 갖는 벡터에 대해 여러 가지 " format "명령어에 따른 출력 양식을 알아 볼 것이다.

```
□format short ↲                % 기본적인 수치표현 양식!
□X=[4/3 1.2345e-6] ↲

X =

    1.3333    0.0000

□format short e ↲
□X ↲

X =

  1.3333e+000  1.2345e-006

□format long ↲
□X ↲

X =

   1.33333333333333   0.00000123450000

□format bank ↲ % 소수 두 번째 자리까지 표시.
□X ↲

X =

          1.33          0.00
```

□format rat ↲ % 분수식 표현!
□X ↲

X =

4/3 1/810045

□format hex ↲ % 16진법에 의한 표현!
□X ↲

X =

3ff5555555555555 3eb4b6231abfd271

□format compact ↲ % 변수의 값을 출력하는 경우 발생하는 많은 blank(공백)을
% 지워준다. 그래서 더 많은 정보를 screen 또는 window에
% 표현할 수 있도록 해준다!
□X ↲
X =
1.3333 0.0000

6. M-files를 작성할 때 이용할 수 있는 특수 값 :

몇몇 함수는 여러분이 M-files를 작성 할 때 유용하게 이용할 수 있는 특수한 값(special values)을 반환한다. 특수 값은 matlab의 " **elmat** " 디렉토리에 위치하고 있다.
다음의 [표 1]은 특수 값을 요약 정리 한 것이다.

[표 1]의 특수 값 중에서 " **computer** "와 " **eps** "에 대한 예제를 다루어 보자.

□[C , MaxSize]=computer ↲

C =

PCWIN

MaxSize = %임의의 한 행렬이 수용할 수 있는 원소 갯수의 최대 값

268435455

" **computer** "라는 함수는 여러분의 computer 형(type)이 무엇인지를 반환함과 동시에, 여러분이 현재 쓰고 있는 matlab버전이 지원할 수 있는 최대 크기의 행렬 원소 갯수를 나타내 준다.



여러분이 matlab command window([그림 1.5])에서 작업하는 경우 잘못 typing하여 "return key "를 치고, 다시 typing하는 경우가 있을 것이다. 이때, matlab은 " editing keys "를 제공하여 여러분의 편집 과정을 돕는다. 다음의 [표 2]는 " editing keys "중 몇 개를 정리하였다. 또한, 한 명령어 라인이 긴 경우가 있을 것이다. 이 경우, " ... "를 사용한다. " ... "는 명령어 라인이 다음 라인에 계속됨을 나타낸다.

```
□s=1 - 1/2 + 1/3 - 1/4  + 1/5 - 1/6  + 1/17 ... ↲   %다음 라인에 계속!
       - 1/8 + 1/9 - 1/10 + 1/11 - 1/12; ↲
```

matlab은 한 개의 라인에서 최대 4096개의 문자를 작성 할 수 있게 한다.

특수 값 반환 함수 및 상수	각각의 특수 값에 대한 설명
eps	matlab이 수치계산 할 때 발생하는 허용 오차 (tolerance)
realmax	여러분의 컴퓨터가 표현 할 수 있는 가장 큰 부동 소수점을 반환한다. (Largest floating-point number)
realmin	여러분의 컴퓨터가 표현 할 수 있는 가장 작은 부동 소수점을 반환한다. (Smallest floating-point number)
pi	3.1415926535897...
i 와 j	허수의 단위(즉, i^2 = -1), 그러므로, 가능하면, i와 j는 변수로 취하지 않는 것이 좋겠다.
inf	무한대(infinity)를 반환한다. ---> (임의의 nonzeros 수치) / 0
NaN	수치가 안임(Not-a-number)을 나타낸다. ---> 0 / 0 또는 inf / inf
computer	여러분의 컴퓨터 형(type)을 반환한다. (예, pcwin 등등)
flops	부동 소수점 연산의 횟수를 반환한다.
version	여러분이 현재 사용하는 matlab의 버전을 반환한다.

표 1 matlab의 특수 값 요약

key symbol	편집 키들에 대한 설명
↑	**이전 명령 라인을 되부른다.** (Recall previous line)
↓	**다음 명령 라인을 되부른다.** (Recall next line)
home	**명령 라인의 처음으로 커서를 옮긴다.** (Move to beginning of line)
end	**명령 라인의 끝으로 커서를 옮긴다.** (Move to end of line)

표 2 matlab 편집 키들의 요약



다음 이 시간에는 기본적인 그래픽을 활용하면서 계속해서 matlab의 행렬 연산을 배우게 될 것이다.